

Local Smoothing

- Overview of Local Smoothing: **Interpolation** vs **Smooth**
- Explain three popular local smoothers for (Y_i, x_i) :
 - **LOESS/LOWESS** (Locally estimated/weighted scatter plot smoothing):
 - **Kernel Smoothing**
 - **Spline Smoothing**

Regression in General

- **Data**: $(Y_i, X_{i1}, \dots, X_{ip})$ for $i = 1, 2, \dots, n$

- The (ordinary) linear regression model:

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \epsilon$$

but this may be too restrictive.

- A more general model:

$$Y = f(X_1, X_2, \dots, X_p) + \epsilon$$

for a broad class of “**smooth**” function f .

Special Case: $p=1$

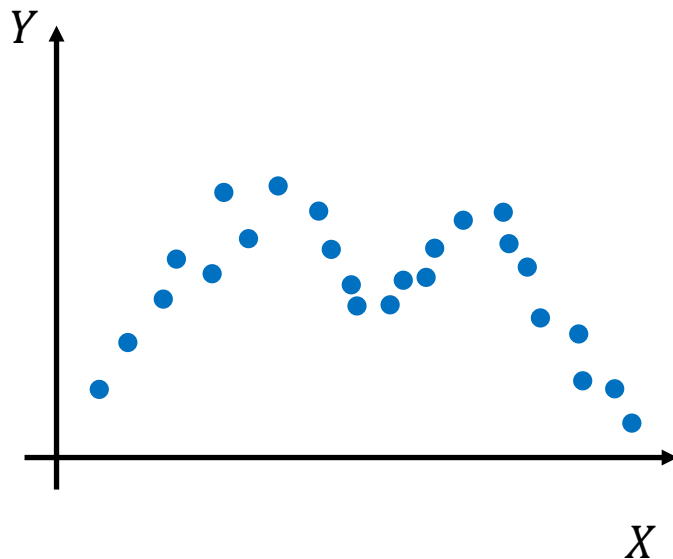
Let us consider the special case of the dimension $p=1$:

Data: $(Y_i, x_i), i = 1, 2, \dots, n.$

Objective: Predict Y at a given $X = x.$

Model: $Y_i = f(x_i) + \epsilon_i$ for some nonlinear function $f(\cdot)$

Question: how to estimate f ?



Method of Least Squares

- By the method of least squares, we want to find a function f to minimize the residuals sum squares

$$RSS = \sum_{i=1}^n \epsilon_i^2 = \sum_{i=1}^n (Y_i - f(x_i))^2$$

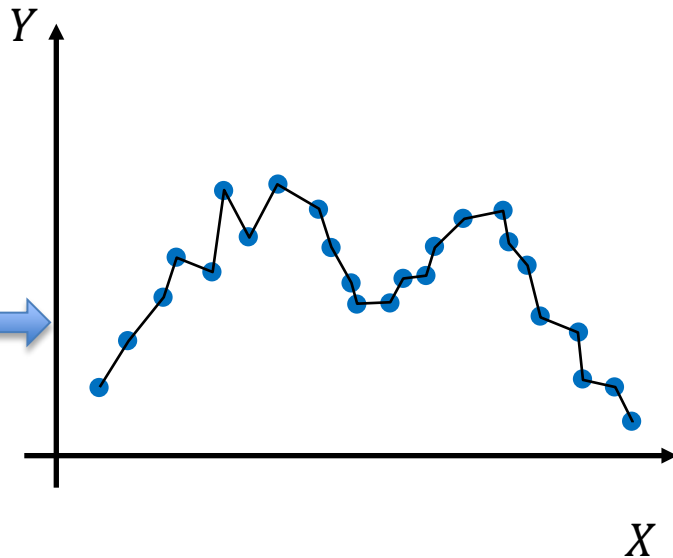
- Find a function f that passes through all points?

Interpolation

Interpolation

Three popular interpolation methods:

- **Piecewise constant**
(dis-continuous)
- **Piecewise linear interpolation**
- **Polynomial Interpolation**
(very smooth)



Weakness of Interpolation

- The interpolation curves are easy to understand and provide a good picture, but they are generally inadequate for prediction or estimation, as it is difficult to control the model complexity.
- Local versus Global Approximation
 - Piecewise constant or linear interpolation is too local, as they only use information from one or two data points
 - Polynomial interpolation uses all information from all data points, but unfortunately it does not work well when the sample size n is large

Transition to Local Smoothers

- **Data:** $(Y_i, x_i), i = 1, 2, \dots, n.$
- **Model:** $Y_i = f(x_i) + \epsilon_i$ for some nonlinear function $f(\cdot)$
- **Interpolation:** assume that all error terms $\epsilon_i \equiv 0.$
- **Local Smoother:**
 - it allows that the error term ϵ_i is non-zero (ideally small),
 - it assumes that $f(\cdot)$ is locally smooth.

Problem Formulation for Local Smoother

- **Data**: (Y_i, x_i) for $i = 1, 2, \dots, n$
- **Model**: $Y_i = f(x_i) + \epsilon_i$ for some nonlinear function $f(\cdot)$, where the error term ϵ_i are independent and identically distributed with mean 0.
- **Question**: can we find an estimate of $f(x)$ at a given x .

Three Popular Local Smoothers

Three popular local smoothers for (Y_i, x_i) :

1. **LOESS/LOWESS** (Locally estimated/weighted scatter plot smoothing): combine weighted least squares with k-nearest neighborhood
2. **Kernel Smoothing**
3. **Spline Smoothing**

1. LOESS Algorithm: High-Level Ideas

LOcally Estimated/Weighted Scatterplot Smoothing (LOESS) Algorithm, developed in Cleveland (1979):

- **k-nearest neighbor:** for a given x , find the k training data points that are closet to x
- **Weighted linear regression:** Use these k data points to fit a low-degree polynomial, often degree of $1 \sim 3$, by locally weighted least squares. The closer an x_i is to x , the more weight it gets.
- $\hat{Y} = \hat{f}(x)$ is the predicted value of this local polynomial at x

LOESS Algorithm: Weighted Least Squares

- 1) At a given x , it defines the tri-cube weight

$$w_i = \begin{cases} \left(1 - \left|\frac{x_i - x}{d_k}\right|^3\right)^3 & \text{if } |x_i - x| \leq d_k \\ 0 & \text{otherwise} \end{cases}$$

where d_k = the k -th smallest distance from any x_i to x .

- 2) It fits the weighted least squares

$$\hat{\beta} = \operatorname{argmin} \sum_{i=1}^n w_i \left(Y_i - (\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3) \right)^2$$

- 3) At the given x , we then use the estimated $\hat{\beta} = \hat{\beta}(x)$ predicts

$$\hat{f}(x) = \hat{\beta}_0 + \hat{\beta}_1 x + \hat{\beta}_2 x^2 + \hat{\beta}_3 x^3$$

Advantage/Weakness of LOESS Algorithm

- Advantage:

- It is nonparametric, as it does not fit a closed-form function
- It works best on large, densely sampled, low-dim data set

- Weakness:

- It is computationally expensive, and needs a lot of data to get good fit
- One does not get a convenient closed-form function, and thus it is difficult to communicate the results to other people
- The LOESS is not intended for high-dimensional data

2. Kernel Smoothing

Data: $(Y_i, x_i), i = 1, \dots, n$

Model: $Y_i = f(x_i) + \epsilon_i, i = 1, \dots, n$

At the high-level, **kernel smoothing** is linear in the sense that

$$\hat{f}(x) = \sum_{i=1}^n w(x_i, x) Y_i$$

where the weights $w(x_i, x)$ are constructed from the kernel $K_h(u)$ to describe the “distance” between x_i and x .

- The shape of the weights comes from the shape of kernel $K(u)$.
- The size of the weights is determined by the bandwidth h .

Nadaraya-Watson Estimator

The Nadaraya-Watson (NW) kernel estimate of $f(x)$ is given by

$$\hat{f}(x) = \sum_{i=1}^n w(x_i, x) Y_i = \frac{\sum_{i=1}^n K_h(x - x_i) Y_i}{\sum_{i=1}^n K_h(x - x_i)}$$

- A kernel $K(u)$ is bounded, continuous function on $(-\infty, \infty)$ satisfying

1) $K(u) \geq 0$

2) $\int_{-\infty}^{+\infty} K(u) du = 1$

- $K_h(u) = \frac{1}{h} K\left(\frac{u}{h}\right)$

- Example: $K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right)$ = pdf of $N(0,1)$ and K_h = pdf of $N(0, h^2)$.

3. Spline Smoothing: Cubic Splines

- Data: $(Y_i, x_i), i = 1, 2, \dots, n$, where $a = x_1 < x_2 < \dots < x_n = b$.
- Model: $Y_i = f(x_i) + \epsilon_i$
- Question: how to estimate the function $f(x)$ at a given $x \in [a, b]$.
- Cubic Splines: **piecewise polynomial of degree 3**.
 - Interpolating, i.e., $Y_i = f(X_i)$ for $i = 1, 2, \dots, n$.
 - Smoothing, i.e., **solution of the penalized optimization problem**

$$\hat{f} = \operatorname{argmin}_{f \in H^2[a,b]} \left[\frac{1}{n} \sum_{i=1}^n (Y_i - f(X_i))^2 + \lambda \int_a^b (f''(t))^2 dt \right]$$

Optimization Algorithm for Cubic Splines

Idea to solve the problem $\hat{f} = \underset{f \in H^2[a,b]}{\operatorname{argmin}} \left[\frac{1}{n} \sum_{i=1}^n (Y_i - f(X_i))^2 + \lambda \int_a^b (f''(t))^2 dt \right]$

- Use the B-spline basis decomposition: $\hat{f}(x) = \sum_{i=1}^{n-d} \beta_i B_{i,d}(x)$
- Restate the original optimization as ridge regression problem:

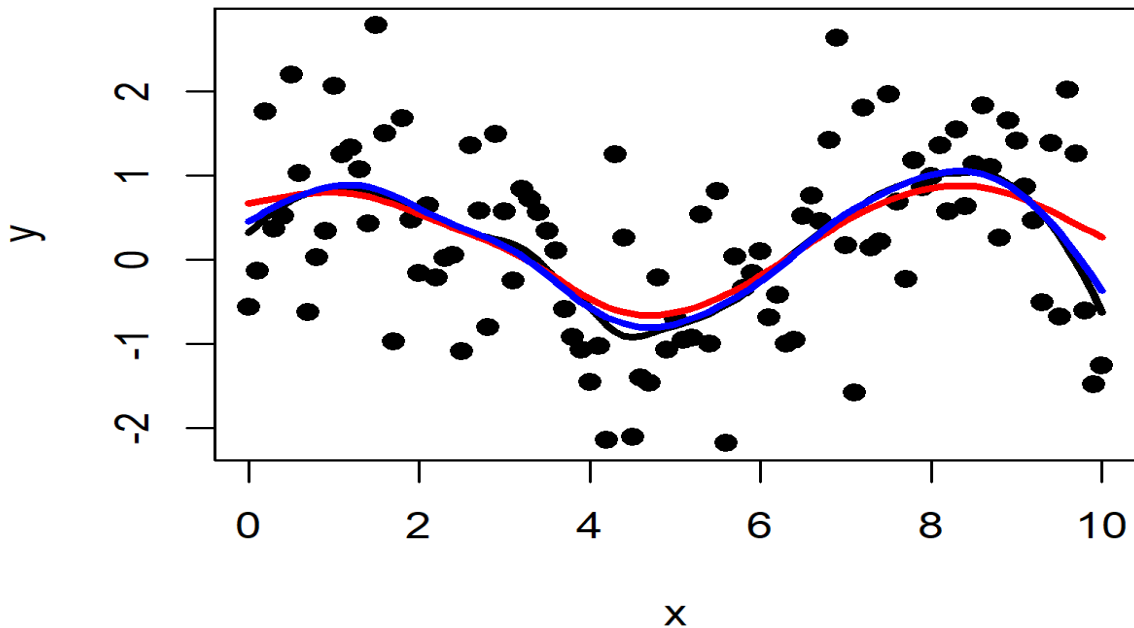
$$\hat{\beta} = \underset{\beta}{\operatorname{argmin}} [|Y - X\beta|^2 + \lambda^* \beta^T V \beta] \quad \text{with } \lambda^* = n\lambda$$

- Taking derivative w.r.t β and setting to 0 yields
$$-2 U^T (Y - U\beta) + 2 \lambda^* V \beta = 0$$
- Thus the optimal solution is

$$\hat{\beta} = (U^T U + \lambda^* V)^{-1} U^T Y$$

A comparison of Local Smoothing Methods

(LOESS-Black, Kernal-Red, Spline-Blue). R codes at BrightSpace



Summary: Local Smooth

- Interpolation: Piecewise Linear
- Local smoothing algorithms:
 1. LOESS/LOWESS
 2. Kernel Smoothing
 3. Spline Smoothing